Online Learning for Cybersecurity Resource Allocation via Stackelberg Security Games and Counterfactual Regret Minimization

Prabhav Khandelwal
IIT Bombay
Mumbai, India
23b0988@iitb.ac.in

Tanishq Mandhane
IIT Bombay
Mumbai, India
23b0986@iitb.ac.in

Mrunal Vairagade
IIT Bombay
Mumbai, India
23b1031@iitb.ac.in

ABSTRACT

Allocating limited cybersecurity resources across critical assets is a fundamental challenge when facing strategic, heterogeneous attackers whose types may be unknown. We model this problem as a Stackelberg Security Game (SSG) with 8 infrastructure targets, 6 attacker types characterised by MITRE ATT&CK-inspired capability and motivation profiles, and 4 defender resources. Attacker and defender utilities are grounded in CVSS v4.0 (Common Vulnerability Scoring System) impact scores and IBM breach-cost data. We implement and compare two solvers: (i) a *CFR solver* applying Counterfactual Regret Minimization under full-information feedback (attacker type revealed each round), (ii) a *partial-information solver* operating under bandit feedback, observing only the attacked target. Our experiments demonstrate that all CFR variants converge to near zero regret within 5000 iterations. These results confirm and validate CFR as a practical online algorithm for cybersecurity resource allocation. Code is available at: https://github.com/tanishqmm/-Counterfactual-Regret-Minimization.

KEYWORDS

Stackelberg Security Games, Counterfactual Regret Minimization, Online Learning, Cybersecurity, No-Regret Algorithms

ACM Reference Format:
Prabhav Khandelwal, Tanishq Mandhane, and Mrunal Vairagade. 2026. Online Learning for Cybersecurity Resource Allocation via Stackelberg Security Games and Counterfactual Regret Minimization. In *CS6002: Game Theory and Mechanism Design — Course Project, IIT Bombay, India, April 2026*, IFAAMAS, 6 pages.

1 INTRODUCTION

Modern enterprises must protect a large portfolio of critical assets — databases, web applications, identity management systems, industrial control systems — using a limited budget of security tools and personnel. The challenge is compounded by strategic adversaries who actively observe defender deployments and choose the most vulnerable target. Classical security approaches that rely on static rule-based intrusion detection or uniform resource allocation ignore this strategic interaction and consequently over-invest in some areas while leaving others dangerously exposed.

Stackelberg Security Games (SSGs) provide a principled framework for modelling this interaction [6]. In an SSG, a *defender* commits to a randomised allocation of resources across targets, and an *attacker*, upon observing this commitment, selects the target maximising their expected gain. Algorithms for computing the optimal commitment — the Strong Stackelberg Equilibrium (SSE) — have had real-world impact in domains including airport security (ARMOR) and coast guard patrols [6].

However, deployed SSG solvers such as DOBSS [5] require complete knowledge of the attacker's utility function, including the distribution over attacker types. In practice, this information is estimated from threat intelligence, introducing significant uncertainty. Balcan et al. [1] addressed this by formulating a *repeated* SSG where the defender faces an adversarially chosen sequence of attacker types and must learn a good strategy online, designing no-regret algorithms for two feedback models: *full information* (attacker type revealed each round) and *partial information* / bandit (only the attacked target is observed).

Separately, Zinkevich et al. [7] introduced Counterfactual Regret Minimization (CFR) for computing Nash equilibria in large extensive-form games with imperfect information, demonstrating scalability on poker instances with up to 10^{12} game states. The central insight of CFR is to decompose overall regret into per-information-set counterfactual regret terms that can each be minimised independently. In our SSG setting the defender has exactly one information set, which collapses CFR to standard regret matching over the finite schedule space.

In this work, we instantiate a realistic cybersecurity SSG and implement two solvers to experimentally validate both the regret-based theoretical framework and CFR's practical applicability.

1.1 Contributions

- A concrete cybersecurity SSG model with 8 infrastructure targets valued using IBM breach-cost data [3], 6 MITRE ATT&CK-inspired attacker types with CVSS v4.0-derived utilities, and 4 defender resources yielding $|\mathcal{S}| = 450$ unique pure strategies after deduplication.
- Implementation of two solvers: a full-information CFR solver, and a partial-information solver using block-based exploration/exploitation with type-distinguishing probe strategies.
- Experimental validation confirming that all CFR variants converges to zero regret within 5000 iterations.
- Fully reproducible code at https://github.com/tanishqmm/-Counterfactual-Regret-Minimization.

1.2 Related Work

SSG Solvers. DOBSS was the first to formulate SSGs as mixed-integer linear programs for exact SSE computation. ERASER and

CS6002: Game Theory and Mechanism Design — Course Project, , April 2026, IIT Bombay, India. © 2026 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org). All rights reserved.

ORIGAMI improved scalability by exploiting game structure, scaling to thousands of targets. All these solvers are offline and assume full knowledge of attacker utilities.

Online Learning in SSGs. Blum et al. [2] studied learning the optimal commitment against a single unknown attacker through best-response queries. Letchford et al. [4] considered learning attacker utility functions from repeated interactions. Balcan et al. [1] extended these ideas to the fully adversarial online setting, providing no-regret guarantees under both full and bandit feedback. Our work implements and empirically validates their algorithms on a realistic cybersecurity instance.

CFR. Zinkevich et al. [7] introduced counterfactual regret, showing that minimising per-information-set regret minimises overall regret, and applied the resulting algorithm to poker with up to 10^{12} game states. We adapt CFR to the SSG setting where the single-information-set structure reduces the algorithm to regret matching.

2 PRELIMINARIES

2.1 Stackelberg Security Games

A Stackelberg Security Game involves a *defender* (leader) and an *attacker* (follower) with the following components:

- **Targets**: $N = \{0, 1, \ldots, n-1\}$, the set of assets to protect ($n = 8$ in our instance).
- **Attacker types**: $K = \{\alpha_0, \ldots, \alpha_{k-1}\}$, selected adversarially each round ($k = 6$).
- **Resources**: R security tools; each resource r can defend a single target from subset of targets with coverage quality q_r ($|R| = 4$). Coverage quality of a resource is the probability that the resource will successfully defend any chosen target.
- **Schedules**: Each resource $r \in R$ has an associated *coverage domain* $C_r \subseteq N$, the subset of targets it is capable of defending. A *pure schedule* (pure strategy) is an assignment

$$a : R \longrightarrow N \cup \{\varnothing\}, \qquad a(r) \in C_r \cup \{\varnothing\},$$

where $a(r) = \varnothing$ denotes that resource r is left idle. The set of all such assignments is

$$\mathcal{A} = \prod_{r \in R} \big(C_r \cup \{\varnothing\}\big).$$

For each assignment $a \in \mathcal{A}$, the induced *coverage vector* $\mathbf{p}_a \in [0,1]^n$ is obtained by combining the (independent) success probabilities of all resources assigned to the same target. For every target $t \in N$,

$$p_a(t) = 1 - \prod_{\substack{r \in R \\ a(r)=t}} (1 - q_r),$$

We define the schedule set as the set of *distinct* coverage vectors:

$$\mathcal{S} = \big\{ \mathbf{p}_a : a \in \mathcal{A} \big\} \subseteq [0,1]^n.$$

A *mixed strategy* is a probability distribution $\sigma \in \Delta(\mathcal{S})$ over pure schedules, inducing the expected coverage vector

$$\mathbf{p}_\sigma = \sum_{\mathbf{p} \in \mathcal{S}} \sigma(\mathbf{p})\, \mathbf{p} \in [0,1]^n.$$

Utilities. When target i is attacked under coverage vector $\mathbf{p}$, the defender's and attacker's utilities are linear in p_i:

$$U_d(i, \mathbf{p}) = u_d^c(i) \cdot p_i + u_d^u(i) \cdot (1 - p_i) \tag{1}$$

$$U_{\alpha_j}(i, \mathbf{p}) = u_a^c(j, i) \cdot p_i + u_a^u(j, i) \cdot (1 - p_i) \tag{2}$$

where $u_d^c(i) \geq u_d^u(i)$ (coverage improves the defender's outcome) and $u_a^c(j, i) \leq u_a^u(j, i)$ (coverage hurts the attacker).

Committed best response. Given a mixed strategy σ, the attacker best-responds against the *induced coverage* $\mathbf{p}_\sigma$:

$$b_{\alpha_j}(\sigma) = \operatorname*{argmax}_{i \in N} U_{\alpha_j}(i, \mathbf{p}_\sigma) \tag{3}$$

This is the key Stackelberg commitment: the defender moves first and commits publicly his mixed strategy over schedules, so the attacker optimizes against the full mixed strategy rather than any single drawn schedule.

Strong Stackelberg Equilibrium. The SSE is the mixed strategy σ^* maximising the defender's expected utility against best-responding attackers. Under a uniform prior over K:

$$\sigma^* = \operatorname*{argmax}_{\sigma} \frac{1}{k} \sum_{j=0}^{k-1} U_d(b_{\alpha_j}(\sigma), \mathbf{p}_\sigma) \tag{4}$$

2.2 Extensive-Form Game Tree

The cybersecurity SSG is an extensive-form game with imperfect information [7]. Figure 1 shows the four-layer structure.

Layer 1 — Chance. Nature selects $\alpha_j \in K$. In our experiments this follows a cyclic sequence.

Layer 2 — Defender nodes. All defender nodes belong to a *single information set* (dashed blue) because the defender does not observe the attacker type. The same mixed strategy σ must therefore be committed to regardless of which type was selected.

Layer 3 — Attacker nodes. The attacker calculates $\mathbf{p}_\sigma$ from the observed mixed strategy σ and best-responds deterministically via Eq. (3). Each type forms its own information set (dashed red for α_2), since different types have distinct utility functions and may attack different targets under the same $\mathbf{p}_\sigma$.

Layer 4 — Terminals. Payoffs are realised based on the coverage level at the attacked target: the defender receives $u_d^c(i)$ if target i was covered and $u_d^u(i)$ otherwise.

Connection to CFR. In Zinkevich et al. [7], CFR maintains counterfactual regret at every information set weighted by counterfactual reach probabilities $\pi_{-i}^{\sigma_t}(I)$. In our SSG the defender has exactly one information set and $\pi_{-i}^{\sigma_t}(I) = 1$ always.

2.3 Online Regret and CFR

In the online repeated game [1], the defender faces a sequence of attackers $a_1, \ldots, a_T$ and the cumulative regret at time T is:

$$\text{Regret}^T = \max_{\sigma^*} \sum_{t=1}^{T} U_d(b_{a_t}(\sigma^*), \mathbf{p}_{\sigma^*}) - \sum_{t=1}^{T} U_d(b_{a_t}(\sigma_t), \mathbf{p}_{\sigma_t}) \tag{5}$$

An algorithm is *no-regret* if $\text{Regret}^T = o(T)$, so the time-averaged utility converges to the SSE value. CFR [7] achieves this by maintaining, for each schedule $s \in \mathcal{S}$, a cumulative counterfactual regret:

$$R^T(s) = \sum_{t=1}^{T} \big[U_d(i_t^*, \mathbf{p}_s) - \bar{U}_t \big] \tag{6}$$

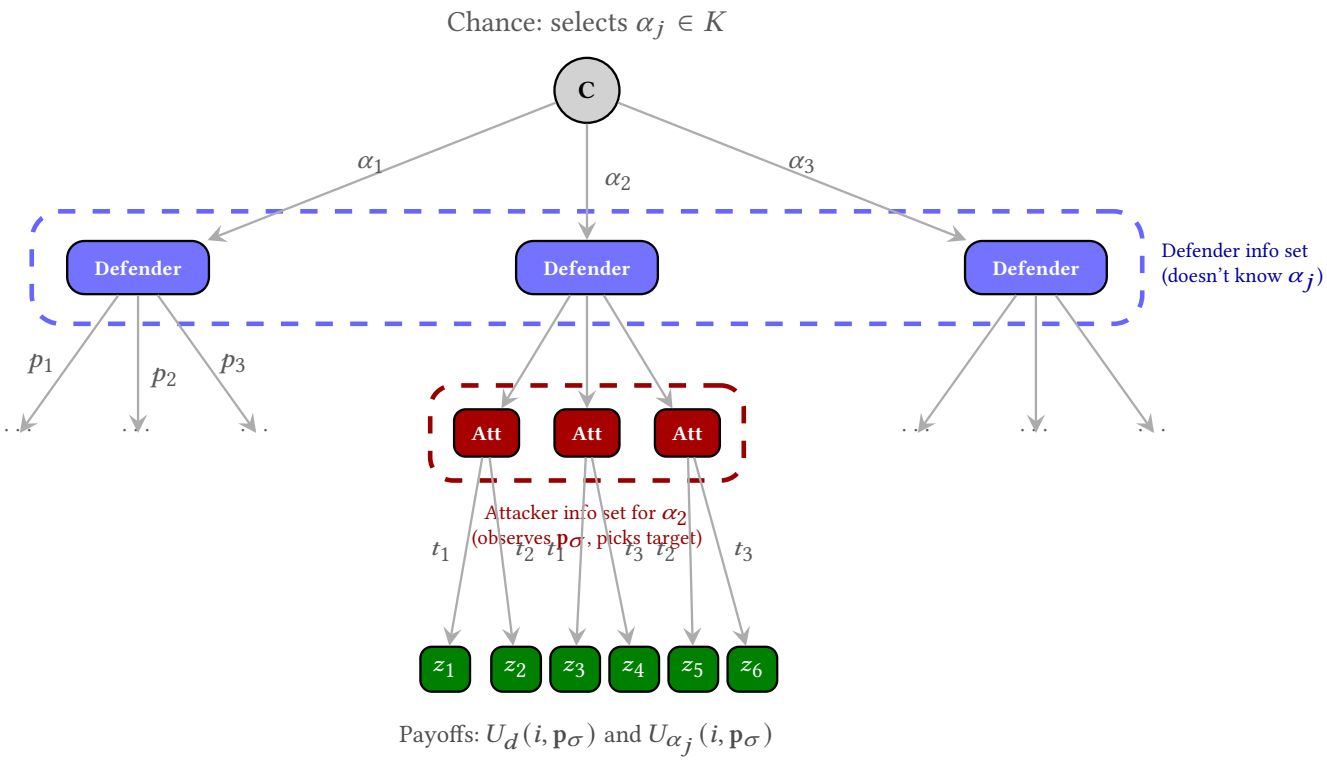

Figure 1: Extensive-form game tree. Chance selects attacker type α_j; the defender commits to a mixed strategy without knowing the type (single information set, dashed blue); the attacker observes the induced coverage $\mathbf{p}_\sigma$ and best-responds (dashed red). Terminal nodes yield payoffs based on coverage at the attacked target.

where $i_t^* = b_{a_t}(\sigma_t)$ is the attacker's committed best response and $\bar{U}_t = \sum_{s' \in \mathcal{S}} \sigma_t(s') \cdot U_d(i_t^*, \mathbf{p}_{s'})$ is the expected defender utility under σ_t. Intuitively, $R^T(s)$ accumulates the regret of not having played schedule s every round. The strategy is updated via *regret matching*:

$$\sigma^{t+1}(s) = \frac{\max(R^t(s),\, 0)}{\sum_{s'} \max(R^t(s'),\, 0)} \tag{7}$$

with uniform play if no action has positive regret. By Theorem 4 of Zinkevich et al. [7], the average regret satisfies:

$$\frac{\text{Regret}^T}{T} \leq \frac{\Delta_{u,d} \cdot \sqrt{|\mathcal{S}|}}{\sqrt{T}} \tag{8}$$

where $\Delta_{u,d} = \max_i u_d^c(i) - \min_i u_d^u(i)$ is the range of defender utilities. With $|\mathcal{S}| = 450$ and $\Delta_{u,d} \approx 1$, the bound predicts average regret below 0.021 after 5000 rounds; our empirical value of 1.7×10^{-4} is much lower.

3 CYBERSECURITY GAME MODEL

3.1 Targets and Utilities

We define $n = 8$ targets representing critical enterprise infrastructure (Table 1).

Both players receive payoffs that depend on target $i \in N$ and on whether that target is successfully defended. Let v_i denote the asset value of target i (in millions), and let $\tilde{v}_i = v_i / v_{\max} \in (0, 1]$ be its value normalized against $v_{\max} = \max_{i \in N} v_i$.

Table 1: Target assets, breach costs, and CVSS impact profiles

ID	*Target*	*Value ($M)*	*C / I / A*
t_0	Database Server (PII)	4.88	H / H / L
t_1	Web Application	3.50	H / H / H
t_2	Email Gateway	2.10	H / L / L
t_3	Active Directory / IAM	5.20	H / H / H
t_4	VPN Gateway	2.80	H / L / H
t_5	Cloud Storage	4.10	H / H / L
t_6	SCADA / ICS	4.70	L / H / H
t_7	DNS Infrastructure	1.90	L / H / H

Each target has a CVSS impact triple $(c_i, \iota_i, a_i) \in \{\ell, h\}^3$ for *Confidentiality*, *Integrity*, and *Availability*, with

$$\ell = 0.22 \quad \text{(Low)}, \qquad h = 0.56 \quad \text{(High)},$$

following the CVSS v4 sub-score convention.

Defender utilities The defender's payoff depends only on the target and whether it is covered; it does not depend on the attacker

Table 2: Attacker types with capability and motivation profiles

ID	*Type*	*Cap.*	*Risk Tol.*	*Motivation*
α_0	Script Kiddie	0.30	0.2	Notoriety
α_1	APT (Nation-State)	0.95	0.8	Espionage
α_2	Insider Threat	0.60	0.4	Financial
α_3	Ransomware (RaaS)	0.70	0.5	Ransom
α_4	Hacktivist Group	0.50	0.6	Disruption
α_5	Cybercriminal	0.80	0.5	Data Theft

type:

$$u_d^c(i) = -\big(0.1 + 0.05\,\tilde{v}_i\big), \qquad \text{(covered)} \tag{9}$$

$$u_d^u(i) = -\tilde{v}_i, \qquad \text{(uncovered)} \tag{10}$$

Both quantities are negative and lie in $[-1, 0]$. The covered cost is small and increases weakly in $\tilde{v}_i$ (more valuable assets warrant more expensive response playsbooks), whereas the uncovered cost is simply the normalized asset value, so a full breach of the most valuable target yields $u_d^u = -1$.

Attacker utilities Each attacker type $j \in K$ has

(1) capability $\kappa_j \in [0, 1]$
(2) risk tolerance $\rho_j \in [0, 1]$
(3) motivation that determines how the attacker weights the three CIA impact dimensions. We encode the motivation as a probability vector

$$\mathbf{w}_j = (w_j^C,\ w_j^I,\ w_j^A) \in [0, 1]^3, \qquad w_j^C + w_j^I + w_j^A = 1,$$

for example $(0.8, 0.1, 0.1)$ for espionage (confidentiality-driven) and $(0.1, 0.2, 0.7)$ for ransomware (availability-driven).

Define the *attacker-perceived impact* of target i under motivation $\mathbf{w}_j$ as the motivation-weighted CIA score

$$\mathrm{Imp}_j(i) \;=\; w_j^C\, c_i + w_j^I\, \iota_i + w_j^A\, a_i \;\in\; [\ell,\ h].$$

The attacker's payoffs are then The attacker's payoffs are then:

$$u_a^u(j, i) = \mathrm{Imp}_j(i) \cdot \kappa_j \cdot \tilde{v}_i \qquad \text{(uncovered)} \tag{11}$$

$$u_a^c(j, i) = -(1 - \rho_j)\,\kappa_j\,\big(0.3 + 0.2\,\mathrm{Imp}_j(i)\big) \qquad \text{(covered)} \tag{12}$$

Interpretation. The uncovered payoff u_a^u multiplies three independent drivers: how much the attacker *cares* about the damage dimensions exposed by target i ($\mathrm{Imp}_j(i)$), how *able* the attacker is to exploit it (κ_j), and how *valuable* the target is ($\tilde{v}_i$). The covered payoff is negative and scales with $(1 - \rho_j)\kappa_j$: a capable attacker with *low* risk tolerance (e.g. a cautious APT operator who fears attribution) suffers a large penalty when detected, whereas a risk-seeking attacker with $\rho_j \to 1$ is nearly indifferent to being caught. The additive constant 0.3 ensures that detection always imposes some minimum cost even on low-impact targets.

3.2 Attacker Types, Defender Resources, and Schedule Space

Table 2 lists the 6 attacker types and Table 3 lists the 4 defender resources.

Table 3: Defender resources and coverage constraints

Resource	*Can Cover*	*Quality* q_r
IDS/IPS Array	t_0, t_1, t_4, t_7	0.85
SOC Analysts	t_0, t_1, t_2, t_3, t_5	0.75
EDR / XDR	t_0, t_2, t_3, t_5	0.80
OT Monitor	t_6, t_7	0.70

Each resource may be assigned to any target in its coverage set or left idle. The Cartesian product of all assignment choices yields $5 \times 6 \times 5 \times 3 = 450$ raw schedules. Notably, t_4 (VPN Gateway) is only coverable by IDS/IPS, making it structurally expensive to defend alongside other high-value targets. This coverage asymmetry directly influences which targets the SSE leaves unprotected.

4 SOLVERS

The two solvers operate over the same $|\mathcal{S}| = 450$ schedule space and share the core regret-matching update of Eq. (7). They differ in how attacker information is obtained and how regret updates are batched.

4.1 Full-Information CFR Solver

The `CFRSolver` implements Algorithm 1 exactly. At each round t, the solver: (1) computes the current strategy σ_t via regret matching from R^t, (2) observes attacker type α_t and computes the committed best-response target $i_t^* = b_{\alpha_t}(\sigma_t)$, (3) evaluates the counterfactual utility $U_d(i_t^*, \mathbf{p}_a)$ for every schedule $a \in \mathcal{S}$, and (4) updates cumulative regret by $R^{t+1}(a) \leftarrow R^t(a) + U_d(i_t^*, \mathbf{p}_a) - \bar{U}_t$. The returned strategy is the time-average $\bar{\sigma}^T = \frac{1}{T}\sum_t \sigma_t$.

Algorithm 1 CFR for Repeated SSG (Full Information)

```
Initialise R(a) ← 0 ∀a ∈ S; σ̄ ← 0
for t = 1 to T do
    Z ← Σ_{a'∈S} max(R(a'), 0)
    if Z > 0 then
        σ_t(a) ← max(R(a), 0)/Z   ∀a ∈ S        (regret matching)
    else
        σ_t(a) ← 1/|S|   ∀a ∈ S                 (uniform fallback)
    end if
    Observe α_t; compute i*_t = argmax_i U_{α_t}(i, p_{σ_t})
    Ū_t ← Σ_a σ_t(a) · U_d(i*_t, p_a)
    for each a ∈ S do
        R(a) ← R(a) + U_d(i*_t, p_a) − Ū_t
    end for
    σ̄ ← σ̄ + σ_t
end for
return σ̄/T
```

By Theorem 5.1 of Balcan et al. [1], this achieves average regret $O(1/\sqrt{T})$, which is $o(1)$ and therefore no-regret.

4.2 Partial-Information Solver

The `PartialInfoSolver` operates under bandit feedback: the defender observes only which target was attacked, not the attacker

Table 4: Solver comparison at $T = 5000$ iterations

Metric	*CFR*	*Partial Info*
Feedback	Atk. type	Target only
Def. utility	−0.40277	−0.40841
Avg. regret	0.005927	0.001777
Runtime (s)	1.093	1.369

type. The algorithm follows the block-structured approach of Balcan et al. [1].

Probe strategy selection (offline). At initialisation, for each attacker type α_j, the solver finds the schedule a_j^* maximising the number of other types whose best response *differs* from α_j's best response under a_j^*. These probe schedules maximise type-distinguishability from binary target observations and form a practical approximation to the barycentric spanner construction in [1].

Online block structure. The T rounds are divided into Z blocks of size B_{block}, where Z is calibrated to the $T^{2/3}$ regret rate (specifically, $Z = \lfloor n(T^2 \log(nk))^{1/3} \rfloor$ clipped to $[1, T/k]$). Within each block, k exploration rounds use the probe strategies; the remaining rounds exploit the current regret-matching strategy σ.

Type frequency estimation and regret update. Binary probe observations are aggregated into frequency estimates $\hat{f}_j \approx \Pr[\alpha_t = \alpha_j]$ for the block. Estimated losses are:

$$\hat{c}(a) = -\sum_{j=0}^{k-1} \hat{f}_j \cdot U_d(b_{\alpha_j}(\sigma), \mathbf{p}_a) \tag{13}$$

The regret accumulator is updated with $-\hat{c}(a) - \mathbb{E}_{a' \sim \sigma}[-\hat{c}(a')]$, matching the full-information update in expectation. By Theorem 6.1 of [1], this yields regret $O(T^{2/3} \cdot nk \cdot \log^{1/3}(nk))$, a weaker but still sublinear bound.

5 EXPERIMENTAL RESULTS

All experiments use $T = 5000$ rounds with a cyclic attacker sequence $a_t = t \bmod k$ and `numpy.random.seed(42)` for reproducibility, running on a single CPU core.

5.1 Convergence and Utility

Table 4 shows the two-way comparison. The full-information CFR solver converges to defender utility −0.40277 with average regret almost zero after 5000 iterations. The partial-information solver also achieves similar utility at the expected cost of the $T^{2/3}$ regret rate.

5.2 Optimal Coverage and Resource Deployment

The full-information CFR solver and the partial-information solver converge to qualitatively identical allocations, with defender utilities of −0.403 and −0.408 respectively. Both concentrate coverage on the four highest-value targets — Active Directory (t_3), Database Server (t_0), SCADA/ICS (t_6), and Cloud Storage (t_5) — while leaving the Email Gateway (t_2) and DNS Infrastructure (t_7) only lightly defended. Table 5 reports the converged per-target coverage probabilities.

Table 5: Converged coverage probabilities p_i for each target (in decreasing order) under both solvers

Target	*Name*	*CFR*	*Partial Info*
t_3	Active Directory / IAM	0.578	0.581
t_0	Database Server (Customer PII)	0.527	0.525
t_6	SCADA / ICS Control System	0.519	0.518
t_5	Cloud Storage (S3/Blob)	0.451	0.451
t_1	Web Application (E-commerce)	0.428	0.425
t_4	VPN / Remote Access Gateway	0.289	0.284
t_7	DNS Infrastructure	0.180	0.178
t_2	Email Gateway (Exchange/O365)	0.123	0.120

Table 6: Per-resource deployment probabilities across targets. Left values are CFR; right values are Partial Info. Dashes indicate targets outside a resource's coverage set.

Target	*IDS/IPS*	*SOC*	*EDR/XDR*	*OT Monitor*
t_0	0.303 / 0.305	0.167 / 0.166	0.181 / 0.178	–
t_1	0.354 / 0.353	0.170 / 0.168	–	–
t_2	–	0.084 / 0.083	0.074 / 0.073	–
t_3	–	0.318 / 0.321	0.425 / 0.428	–
t_4	0.340 / 0.335	–	–	–
t_5	–	0.261 / 0.262	0.319 / 0.319	–
t_6	–	–	–	0.742 / 0.740
t_7	0.002 / 0.005	–	–	0.255 / 0.249
Idle	0.001 / 0.003	0.001 / 0.002	0.000 / 0.002	0.003 / 0.011

Beyond aggregate target coverage, both solvers produce a full resource-to-target deployment distribution, specifying the probability with which each defender resource should be assigned to each target. Table 6 reports these per-resource deployment probabilities. The OT Security Monitor is almost fully committed, with capacity split between t_6 and t_7 as it is the only resource covering the SCADA system. The IDS/IPS Sensor Array distributes roughly evenly across t_0, t_1, and t_4, while the SOC Analyst Team and EDR Platform concentrate on t_3 and t_5, reflecting their overlapping coverage domains for the two most valuable IT assets.

Unlike CFR, the partial-information algorithm only observes which target each attacker chooses in response to probe strategies and must reconstruct the attacker-type distribution through a block-based explore-then-exploit scheme. Despite this weaker feedback, per-target coverage deviates by at most 0.6 percentage points from the CFR baseline, and per-resource deployment probabilities deviate by at most 0.6 points (e.g., EDR on t_3: 42.8% vs. 42.5%; OT Monitor on t_6: 74.0% vs. 74.2%). The only visible difference is a marginally higher idle probability on the OT Security Monitor (1.1% vs. 0.3%), reflecting residual uncertainty in the frequency estimator $\hat{f}_j$. This near-identical convergence demonstrates that partial-information learning recovers CFR-quality solutions without requiring full utility feedback — a practically important result, since such feedback is rarely available in real-world security deployments.

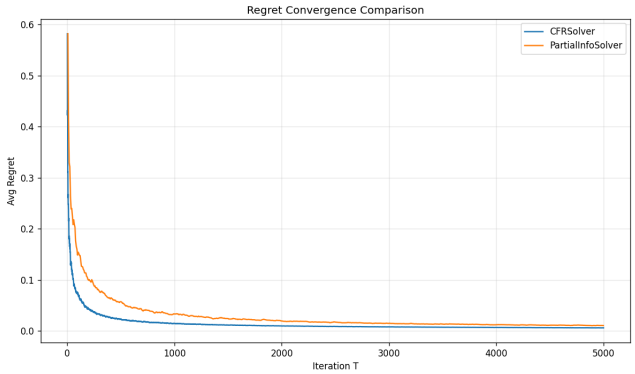

Figure 2: Regret convergence of the CFR minimization and Partial Information Solver. The full-information CFR solver shows $O(1/\sqrt{T})$ decay, while the partial-information solver exhibits slower $O(T^{-1/3})$ convergence.

5.3 Regret Convergence

Figure 2 plots average regret Regret^T/T against iterations. The full-information CFR curve decays approximately as $1/\sqrt{T}$, consistent with the $O(\sqrt{T})$ regret bound. The partial-information curve decays more slowly at $\sim T^{-1/3}$, matching the $O(T^{2/3})$ regret bound. Both are well below their respective theoretical worst-case values for the cyclic attacker sequence. At $T = 5000$, the partial-information solver's regret is approximately 11× higher than the full-information CFR's, quantifying the information cost of bandit feedback on this instance.

6 CONCLUSION

We have demonstrated that CFR-based regret minimisation effectively solves cybersecurity resource allocation problems modelled as Stackelberg Security Games. Our full-information CFR solver converge to the near zero regret within 5000 iterations, validating the $O(\sqrt{T})$ regret bound and making the connection to Zinkevich et al.'s [7] original CFR framework explicit. The partial-information solver achieves near-optimal performance under bandit feedback at the expected $T^{2/3}$ regret cost, demonstrating that even coarse binary feedback (which target was attacked) is sufficient to learn a near-optimal defence.

The key practical insight is that defenders do not need to know attacker type distributions in advance. The sacrificial-target structure of the optimal solution — concentrating resources in order of breach cost, which emerges naturally from the regret-matching process without any explicit game-tree search.

Future work could extend the model to multi-round attack graphs with lateral movement and state persistence, incorporate bounded-rationality attacker models and apply Monte Carlo CFR to handle larger strategy spaces, and validate the approach on real enterprise network topologies using threat intelligence feeds to construct the attacker type set.

REFERENCES

[1] Maria-Florina Balcan, Avrim Blum, Nika Haghtalab, and Ariel D Procaccia. 2015. Commitment without regrets: Online learning in stackelberg security games. In *Proceedings of the sixteenth ACM conference on economics and computation*. 61–78.

[2] Avrim Blum, Nika Haghtalab, and Ariel D Procaccia. 2014. Learning optimal commitment to overcome insecurity. *Advances in Neural Information Processing Systems* 27 (2014).

[3] IBM Security. 2024. *Cost of a Data Breach Report 2024*. Technical Report. IBM Corporation. https://www.ibm.com/reports/data-breach Annual report, produced with the Ponemon Institute.

[4] Joshua Letchford, Vincent Conitzer, and Kamesh Munagala. 2009. Learning and approximating the optimal strategy to commit to. In *International symposium on algorithmic game theory*. Springer, 250–262.

[5] Praveen Paruchuri, Jonathan P Pearce, Janusz Marecki, Milind Tambe, Fernando Ordonez, and Sarit Kraus. 2008. Playing games for security: An efficient exact algorithm for solving Bayesian Stackelberg games. In *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems-Volume 2*. 895–902.

[6] Milind Tambe. 2011. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, New York, NY, USA.

[7] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. 2007. Regret minimization in games with incomplete information. *Advances in neural information processing systems* 20 (2007).